

Spółeczna Akademia Nauk

Instytut Technologii Informatycznych

Kierunek studiów:

INFORMATYKA

Przedmiot:

Wybrane środowiska programowania

Projekt strony internetowej

Adam Muras

Nr albumu: 93289

Grupa: 4

Prowadzący:

mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1	Wprowadzenie	2
2	Analiza technologii i pojęć	3
3	Wybór i opis środowiska programistycznego	6
3.1	Charakterystyka wybranego środowiska	6
3.2	Projekt praktyczny	6
3.3	Dokumentacja projektu	6
4	Wnioski	10
	Bibliografia	12

1 Wprowadzenie

Celem niniejszego projektu dla **juniora** inżyniera jest zaprojektowanie oraz pełne wdrożenie interaktywnej strony internetowej dla profesjonalnego gabinetu psychoterapeutycznego „Ośrodek Równowagi”. Wyzwaniem podjętym w ramach realizacji projektu było stworzenie responsywnego interfejsu użytkownika (UI) oraz zaprogramowanie logiki biznesowej klienta (frontend) z pominięciem ciężkich frameworków JavaScript (takich jak React czy Angular), opierając się wyłącznie na natywnych technologiach przeglądarkowych (Vanilla Stack).

Problem biznesowy automatyzacji procesu rejestracji pacjentów został rozwiązany poprzez implementację autorskiego silnika kalendarza rezerwacyjnego. Z kolei wyeliminowanie kosztów utrzymania dedykowanego serwera backendowego osiągnięto poprzez zastosowanie podejścia Serverless i integrację formularza z zewnętrznym interfejsem programistycznym (API).

2 Analiza technologii i pojęć

W niniejszym rozdziale przeprowadzono szczegółową analizę podstawowych pojęć oraz technologii związanych z inżynierią oprogramowania oraz współczesnymi środowiskami programistycznymi.

- **Programowanie**

Proces projektowania, tworzenia, testowania i utrzymywania kodu źródłowego programów komputerowych. Służy do automatyzacji zadań oraz rozwiązywania złożonych problemów obliczeniowych za pomocą optymalnie dobranych algorytmów [1].

- **IDE (Integrated Development Environment)**

Zintegrowane środowisko programistyczne łączące w jednej aplikacji zaawansowany edytor kodu, kompilator lub interpreter, debugger oraz narzędzia automatyzacji budowania projektu. Służy do maksymalizacji wydajności pracy inżyniera oprogramowania [2].

- **SDK (Software Development Kit)**

Zestaw narzędzi deweloperskich, gotowych bibliotek programistycznych, dokumentacji technicznej oraz kodów przykładowych dostarczany przez producentów oprogramowania. Służy jako kompletne środowisko ułatwiające budowanie aplikacji na określonej platformie [1].

- **JDK (Java Development Kit)**

Specjalistyczny zestaw narzędzi programistycznych dedykowany dla platformy Java. Zawiera m.in. kompilator (javac) oraz środowisko uruchomieniowe (JRE), służąc bezpośrednio do wytwarzania systemów klasy enterprise oraz aplikacji biznesowych [1].

- **Inżynieria oprogramowania**

Dyscyplina techniczna i naukowa stosująca systematyczne, mierzalne oraz rygorystyczne podejście do procesów projektowania, wytwarzania, eksploatacji i późniejszego utrzymania systemów komputerowych [1].

- **Techniki programowania**

Sformalizowany zbiór metod, konwencji strukturalnych oraz sprawdzonych wzorców projektowych stosowanych podczas implementacji kodu źródłowego. Służą do optymalizacji logiki aplikacji, czytelności kodu i ułatwienia jego skalowalności [3].

- **Języki programowania**

Sztuczne, sformalizowane systemy zapisu instrukcji dla maszyn obliczeniowych posiadające ściśle zdefiniowane reguły składniowe (syntaktykę) i znaczeniowe (semantykę). Służą do precyzyjnego opisywania zachowań systemów (np. HTML5, CSS3 czy JavaScript) [3, 4].

- **Języki ezoteryczne**

Języki programowania zaprojektowane jako eksperymenty myślowe, dzieła sztuki programistycznej lub wyzwania intelektualne o celowo nieczytelnej składni. Służą głównie badaniu teoretycznych granic możliwości kompilatorów oraz maszyn Turinga [knuth1984].

- **VPL (Visual Programming Language)**

Wizualny język programowania, w którym logika działania systemu jest definiowana poprzez intuicyjne manipulowanie elementami graficznymi, blokami lub diagramami zamiast pisania tekstu. Służy uproszczeniu modelowania procesów algorytmicznych [1].

- **No-Code / Low-Code**

Platformy i środowiska wytwórcze pozwalające na budowanie aplikacji za pomocą wizualnych konfiguratorów, z minimalnym (Low-Code) lub zerowym (No-Code) udziałem tradycyjnego pisania kodu tekstowego. Służą drastycznemu przyspieszeniu automatyzacji procesów [1].

- **Biblioteka**

Zorganizowany pakiet reużywalnych klas, funkcji lub struktur danych, który programista może zaimportować do swojego projektu. Służy do wykonywania specyficznych zadań pomocniczych bez konieczności ponownego pisania gotowych algorytmów [3].

- **Framework**

Kompleksowy szkielet architektoniczny determinujący strukturę i przepływ sterowania w aplikacji. Narzuca zasady organizacji kodu i służy jako kompletna baza do szybkiej budowy zaawansowanych systemów informatycznych [1].

- **API (Application Programming Interface)**

Interfejs programistyczny aplikacji stanowiący ustrukturyzowany zbiór reguł, metod i protokołów komunikacyjnych. Służy do bezpiecznej wymiany danych i integracji pomiędzy niezależnymi komponentami lub zewnętrznymi serwisami chmurowymi (np. Formspre) [5].

- **Paradygmat programowania**

Fundamentalny styl i powiązana z nim filozofia organizacji kodu źródłowego, determinująca podejście architektoniczne do wykonywania obliczeń oraz modelowania struktur danych (np. paradygmat obiektowy lub funkcyjny) [1].

- **CASE (Computer-Aided Software Engineering)**

Zestaw zautomatyzowanych narzędzi komputerowych wspierających inżynierów na przestrzeni całego cyklu życia oprogramowania. Służą do automatyzacji faz analitycznych, generowania szkieletów kodu oraz projektowania baz danych [1].

- **Modelowanie systemów informatycznych**

Proces tworzenia abstrakcyjnych, sformalizowanych reprezentacji struktury logicznej oraz zachowania systemu. Służy do precyzyjnego mapowania wymagań biznesowych na architekturę techniczną przed etapem kodowania [1].

- **UML (Unified Modeling Language)**

Zunifikowany język modelowania będący ustandaryzowaną notacją graficzną. Służy do wizualizacji, specyfikowania i dokumentowania komponentów systemowych oraz asynchronicznych przepływów danych za pomocą dedykowanych diagramów struktury i zachowania [1].

- **BPMN (Business Process Model and Notation)**

Graficzny standard oparty na diagramach blokowych, przeznaczony do precyzyjnego mapowania procesów biznesowych. Służy jako pomost komunikacyjny ułatwiający zrozumienie procedur operacyjnych organizacji przez analityków i programistów [1].

- **BPM (Business Process Management)**

Systematyczne podejście zarządcze ukierunkowane na identyfikowanie, projektowanie, wykonywanie, kontrolowanie oraz ciągłą optymalizację kluczowych procesów biznesowych zachodzących wewnątrz struktury przedsiębiorstwa [1].

- **CMS (Content Management System)**

System zarządzania treścią w postaci aplikacji webowej z graficznym interfejsem. Służy użytkownikom końcowym do sprawnego dodawania, edytowania oraz publikowania zawartości na stronach internetowych bez wiedzy programistycznej [**website_example**].

- **CMF (Content Management Framework)**

Zaawansowana platforma programistyczna łącząca elastyczność i swobodę architektoniczną czystego frameworka webowego z gotowymi modułami bazy danych systemu CMS. Służy do budowania dedykowanych, wysoce spersonalizowanych portali [**website_example**].

- **WASM (WebAssembly)**

Niskopoziomowy, binarny format instrukcji zaprojektowany jako bezpieczny cel kompilacji dla języków wysokiego poziomu (C, C++, Rust). Służy do uruchamiania wymagających algorytmów obliczeniowych bezpośrednio w przeglądarce z prędkością zbliżoną do kodu natywnego [4].

3 Wybór i opis środowiska programistycznego

3.1 Charakterystyka wybranego środowiska

Decyzja o wyborze natywnego stosu technologicznego (Vanilla Stack) połączonego ze środowiskiem **Visual Studio Code** podyktowana była rygorystycznymi wymogami dotyczącymi wydajności pamięciowej aplikacji. Środowisko to zapewnia pełną kontrolę inżynierską nad przepływem kompilacji "w locie" (JIT - Just-In-Time) na poziomie silnika przeglądarki V8, eliminując narzuty (overhead) generowane przez wirtualny DOM znany z frameworków typu React.

Zalety zastosowanego podejścia to:

- Zminimalizowany rozmiar paczki końcowej (zero-dependency).
- Optymalizacja pod kątem wskaźników Core Web Vitals ze względu na bezpośrednią modyfikację węzłów DOM.
- Modularność i skalowalność kodu uzyskiwana dzięki standardom ES6 Modules.

Do ograniczeń należy zaliczyć wydłużony czas developmentu (Time-To-Market) skomplikowanych komponentów (np. kalendarzy, okien modalnych), które w gotowych frameworkach zazwyczaj importuje się jako gotowe biblioteki.

3.2 Projekt praktyczny

Zaimplementowany system stanowi tzw. Single Page Application o strukturze warstwowej:

1. **Warstwa prezentacji:** Oparta w pełni na technologii CSS Grid, gwarantująca brak załamań układu (Layout Shifts) na urządzeniach mobilnych.
2. **Moduł interaktywny (Akordeon):** Algorytm w czasie rzeczywistym odczytujący własności `scrollHeight` elementów i dynamicznie manipulujący parametrem wysokości, co zapewnia płynne przejścia bez obciążania głównego wątku przeglądarki (Main Thread).
3. **Moduł integracji API:** Uruchamiany po wyborze daty. Silnik waliduje poprawność wprowadzonych danych, serializuje je do obiektu `FormData`, a następnie asynchronicznie przesyła do endpointu *Formspree*, komunikując status operacji użytkownikowi w oknie modalnym.

3.3 Dokumentacja projektu

Interfejs użytkownika został zaprojektowany z myślą o maksymalnej użyteczności i spójności wizualnej. Jednym z kluczowych elementów warstwy prezentacyjnej jest moduł transparentnego cennika (rysunek 1). Został on zaimplementowany z wykorzystaniem modułu CSS Grid,

co gwarantuje stabilne zachowanie układu kafelkowego niezależnie od rozdzielczości ekranu. Karty usług posiadają zaimplementowane płynne transformacje przestrzenne aktywowane po najechaniu kursorem (efekt hover).



Rysunek 1: Warstwa prezentacyjna usług – moduł cennika zaimplementowany w technologii CSS Grid.

Trzon logiczny aplikacji stanowi silnik kalendarza rezerwacyjnego. Algorytm inicjowany jest pobraniem bieżącej instancji czasu z urządzenia użytkownika. Następnie wylicza on indeks ofsetowy pierwszego dnia miesiąca oraz całkowitą liczbę dni w miesiącu (włączając w to detekcję lat przestępnych). Obliczone wartości poddawane są pętli iteracyjnej, która imperatywnie wstrzykuje nowe komórki `<td>` do matrycy tabeli. Wygenerowany w ten sposób, w pełni interaktywny widok kalendarza, przedstawiono na rysunku 2.

Każda z komórek reprezentująca poprawną datę zostaje opieczetowana dedykowanym, izomorficznym formatem czasu w atrybucie danych (np. `data-date="2026-05-30"`). Kliknięcie wybranej komórki wywołuje na ekranie dedykowane okno modalne, aktywując proces formowania ładunku żądania HTTP (rysunek 3). Wewnątrz okna znajduje się formularz walidujący poprawność wprowadzonych danych, które następnie są serializowane do obiektu `FormData` i przesyłane do zewnętrznego interfejsu API.

Zarezerwuj Termin

Wybierz dogodną datę z kalendarza poniżej.

« Maj 2026 »						
Pn	Wt	Śr	Czw	Pt	Sb	Nd
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

Rysunek 2: Autorski silnik kalendarza z możliwością wyboru terminu wizyty.

×

Potwierdź rezerwację

Wybrana data: 2026-05-29

Imię i nazwisko

Adres e-mail

Numer telefonu

Wybierz usługę

Konsultacja Wstępna (180 zł)

▼

Zapisz się na wizytę

Rysunek 3: Asynchroniczny formularz rejestracji w oknie modalnym sprzężony z zewnętrznym API.

4 Wnioski

Wytworzenie i zintegrowanie systemu z zewnętrzną usługą Formspree ukazało potencjał architektury bezserwerowej (Serverless) dla mikro-aplikacji oraz stron wizytówkowych. Pozwoliło to na drastyczne obniżenie kosztów projektowych i infrastrukturalnych, przenosząc ciężar autoryzacji sesji mailowej na certyfikowane oprogramowanie chmurowe.

Największym napotkanym problemem inżynierskim było zapewnienie synchronizacji stanów pomiędzy asynchroniczną odpowiedzią z serwera (obiekty `Promise`) a reakcją graficznego interfejsu (ukrywanie formularza, wyświetlanie komunikatu sukcesu). Błąd ten rozwiązano wdrażając bloki `try...catch` połączone z funkcjami asynchronicznymi `async/await`. Projekt pozwolił na głęboką weryfikację wiedzy z zakresu komunikacji klient-serwer oraz mechanizmów renderowania przeglądarek, potwierdzając przewagę natywnego stosu w projektach o rygorystycznych wymogach optymalizacyjnych.

Spis rysunków

1	Warstwa prezentacyjna usług – moduł cennika zaimplementowany w technologii CSS Grid.	7
2	Autorski silnik kalendarza z możliwością wyboru terminu wizyty.	8
3	Asynchroniczny formularz rejestracji w oknie modalnym sprzężony z zewnętrznym API.	9

Bibliografia

- [1] I. Sommerville. *Inżynieria Oprogramowania*. Warszawa: Wydawnictwo Naukowe PWN, 2020.
- [2] Microsoft. *Visual Studio Code Documentation*. URL: <https://code.visualstudio.com/docs> (term. wiz. 30.05.2026).
- [3] D. Flanagan. *JavaScript. Przewodnik definitywny*. Gliwice: Helion, 2021.
- [4] Mozilla Developer Network. *MDN Web Docs — HTML, CSS, and Web APIs*. URL: <https://developer.mozilla.org> (term. wiz. 30.05.2026).
- [5] Formspree Inc. *Formspree Developer Documentation*. URL: <https://formspree.io/docs/> (term. wiz. 30.05.2026).